VOXI AAC COMMUNICATION PLATFORM

PROJECT TOOLS AND IMPLEMENTATION REPORT

Report date: May 3, 2026

This report documents the complete toolset, frontend implementation, backend implementation, database design, media handling, speech features, OCR flow, chat assistant, deployment setup, and verification checklist for the current Voxi project.

01 PROJECT IDEA AND SYSTEM OVERVIEW

Voxi is an AAC communication web application built to help users express needs, feelings, routines, and messages through visual icons, written expressions, recorded audio, generated speech, drawing recognition, speech practice, emergency contacts, and an AAC assistant chatbot.

The core idea is to replace a single text input with a guided communication system. Users navigate through main categories, choose icons and sub-icons, build simple sentences, and play them as speech. The project combines human recordings, browser speech, AI-generated voices, OCR, speech recognition, and stored communication routines to support different communication needs.

Main Goals

HELP NON-VERBAL OR SPEECH-DELAYED USERS COMMUNICATE

The application gives users visual cards with words, expressions, images, and audio so they can communicate faster without typing full sentences.

ORGANIZE COMMUNICATION BY REAL LIFE CATEGORIES

Content is grouped into main categories, time periods, icons, sub-icons, and sub-sub-icons so users can move from a broad topic to a precise phrase.

SUPPORT MULTIPLE SPEECH OUTPUT MODES

The app can play human recordings, browser speech synthesis, Google TTS, and ElevenLabs AI voices depending on the selected mode and available audio.

SUPPORT TRAINING AND LEARNING

Speech training allows users to practice a word or phrase, compare recognized speech to the target phrase, and track improvement.

SUPPORT DRAWING-TO-SPEECH

Users can draw text or a simple expression, run OCR, edit the recognized phrase, and speak it as audio.

SUPPORT CAREGIVERS AND AAC USERS

The AAC Assistant answers practical AAC questions using a local knowledge base or an optional Groq API provider.

High Level User Flow

1. User opens Voxi and logs in or signs up.
2. User enters Main Categories.
3. User selects a category such as Food, Feelings, Emergency, Training, Daily Routine, Drawing, or AAC Assistant.
4. User navigates through icons, sub-icons, and sub-sub-icons.
5. User selects one or more communication items.
6. Frontend builds a sentence using time words, connectors, parent icon text, and selected item expressions.
7. User chooses a voice mode: human recordings, browser male/female voice, or AI male/female voice.
8. The app plays the selected audio or generates speech.
9. Repeatedly used items are tracked locally and can appear in Daily Routine.
10. Additional support modules handle emergency numbers, speech training, drawing OCR, and chat assistance.

02 TOOLS AND TECHNOLOGY STACK

This module lists the main tools used in the project and what each one does.

Frontend Tools

TOOL	VERSION / SOURCE	ROLE
React	19.2.1	Component-based UI framework
React DOM	19.2.1	Mounts the React app into the browser
Create React App	react-scripts 5.0.1	Development, build, and testing scripts
React Router DOM	7.11.0	Page routing and URL parameters
Bootstrap	5.3.8	Base responsive styling
React Bootstrap	2.10.10	UI components such as buttons, cards, forms, modals, containers, rows, and columns
Fetch API	Browser native	API requests to the Express backend
HTML5 Audio API	Browser native	Audio playback for recordings and generated blobs
Web Speech API	Browser native	Browser speech synthesis and speech recognition
MediaRecorder API	Browser native	Audio recording in speech training
Canvas API	Browser native	Drawing input, image compression, and OCR image creation
File Input Capture	Browser native	Camera and microphone capture on mobile
localStorage	Browser native	Persisted user session, daily routine counts, card order, and training attempts
React Testing Library	package.json	Basic frontend test utilities
Web Vitals	package.json	Frontend performance metrics helper
Vercel	frontend/vercel.json	Frontend deployment rewrite configuration

Backend Tools

TOOL	VERSION / SOURCE	ROLE
Node.js	Runtime	JavaScript backend runtime
Express	5.2.1	REST API server and static file serving
CORS	2.8.5	Allows frontend origins to access backend APIs
Prisma Client	6.19.1	Database ORM for models and queries
Prisma CLI	6.19.1	Migrations and database seeding
PostgreSQL	schema.prisma	Main relational database
Multer	2.1.1	File uploads for images and audio
node-fetch	3.3.2	Backend HTTP calls to download files and call AI providers
google-tts-api	2.0.2	Google TTS audio generation
ElevenLabs API	REST API	AI text-to-speech voice generation
Tesseract.js	7.0.0	OCR engine for drawing recognition
@tesseract.js-data/ara	1.0.0	Arabic OCR language data
@tesseract.js-data/eng	1.0.0	English OCR language data
pngjs	7.0.0	PNG parsing and preprocessing before OCR
crypto.scrypt	Node native	Password hashing with salt
fs/path/url/util	Node native	File paths, static assets, environment loading, and helpers
Groq API	Optional env config	External chatbot provider if CHATBOT_PROVIDER=groq
Railway	Deployed backend URL	Backend hosting target referenced by the frontend

Installed But Not Currently Wired In Visible Source

TOOL	STATUS

body-parser	Installed, but Express JSON middleware is used instead
@iamtraction/google-translate	Installed, but no active backend import was found
@vitalets/google-translate-api	Installed, but no active backend import was found
google-translate-open-api	Installed, but no active backend import was found
say	Installed, but no active backend import was found
uuid	Installed, but no active backend import was found

Not Present In Visible Source But Relevant To TTS Discussion

TOOL	STATUS
ResponsiveVoice	Not installed in package.json and no responsiveVoice script/import was found. The current frontend uses the browser-native Web Speech API instead.

Speech, AI, and Media Tool Benefit Matrix

TOOL	USED WHERE	EXACT BENEFIT IN THIS PROJECT
Human audio recordings	backend/public + frontend pl	ayback Provides the clearest fixed pronunciation for common AAC words and phrases without generating audio every time.
HTML5 Audio API	frontend/src/api and pages	Plays human recordings and generated MP3 blobs returned from backend TTS endpoints.
Web Speech API	frontend/src/api and Trainin	gPage Provides browser speech synthesis for male/female modes and browser speech recognition for pronunciation training.
MediaRecorder API	TrainingPage and SubIconsPag	e Captures the user's spoken practice attempts and supports audio recording/upload for new communication cards.
google-tts-api	backend/index.js	Generates MP3 speech for drawing OCR output and AAC Assistant replies through /api/tts/gtts.
ElevenLabs API	backend/index.js	Generates higher-quality AI voice output through /api/tts/speak using ELEVENLABS_API_KEY and

		mapped male/female voice IDs.
Groq API	backend/index.js	Optional external LLM provider for AAC Assistant answers when CHATBOT_PROVIDER=groq and GROQ_API_KEY are configured.
Local AAC knowledge base	backend/index.js	Keeps the chatbot usable even when Groq is not configured by matching messages to built-in AAC guidance.
Tesseract.js	backend/index.js	Reads handwritten/drawn text from the Express Drawing page and returns recognized text.
Tesseract Arabic/English data	backend/package.json	Gives OCR language support for Arabic and English drawing recognition.
pngjs	backend/index.js	Preprocesses PNG canvas images before OCR to improve recognition quality.
Multer	backend/index.js	Stores uploaded or captured images/audio under backend/public/uploads.
node-fetch	backend/index.js	Calls ElevenLabs/Groq and downloads remote media URLs into local uploads.
Prisma + PostgreSQL	backend/prisma	Persists users, categories, icons, nested icons, emergency numbers, and speech attempts.
Bootstrap + React Bootstrap	frontend pages	Builds responsive grids, cards, forms, modals, and layout without custom UI infrastructure.
localStorage	AppContext and utilities	Saves the logged-in user, daily routine counts, card ordering, and local training attempts in the browser.

03 FRONTEND APPLICATION LAYER

The frontend is a React application that provides routing, page layout, user session state, category navigation, icon selection, speech controls, emergency tools, training, drawing OCR, and chat assistant screens.

Capabilities

GLOBAL ROUTING

App.js defines all application routes using React Router DOM.

GLOBAL USER STATE

AppContext stores the logged-in user in localStorage and exposes saveUser, logout, and isLoggedIn.

API SERVICE LAYER

api.js centralizes API base URL selection, media URL normalization, speech helpers, TTS requests, chat requests, and OCR requests.

AUTH SERVICE LAYER

auth.js wraps login and signup requests and parses backend errors.

RESPONSIVE UI COMPONENTS

Bootstrap and React Bootstrap provide cards, containers, forms, buttons, modals, grids, rows, and columns.

BROWSER CAPABILITY INTEGRATION

The frontend uses speechSynthesis, SpeechRecognition, MediaRecorder, canvas, Audio, file uploads, camera capture, microphone capture, and localStorage.

Source Files

FILE	LOCATION	ROLE
App.js	frontend/src/	Defines the full route map
index.js	frontend/src/	Mounts React in StrictMode
AppContext.js	frontend/src/context/	Global user session context
api.js	frontend/src/api/	API base URL, media normalization, TTS, chat, OCR, browser speech helpers
auth.js	frontend/src/api/	Login and signup API wrapper
Landing.js	frontend/src/pages/	Landing page and navigation entry point
MainCategoriesPage.js	frontend/src/pages/	Loads main categories and sends users to category-specific routes
IconsPage.js	frontend/src/pages/	Displays icons for a main category or time period

SubIconsPage.js	frontend/src/pages/	Selection, sentence building, audio playback, upload modal, ordering, and content creation
SubSubIconsPage.js	frontend/src/pages/	Nested item selection, search, sentence building, and playback
SubIconDetail.js	frontend/src/pages/	Single sub-icon detail and speak action
SubSubIconDetail.js	frontend/src/pages/	Single sub-sub-icon detail, volume, speed, and speak action
TrainingPage.js	frontend/src/pages/	Speech recognition, recording, scoring, and progress
ExpressDrawingPage.js	frontend/src/pages/	Canvas drawing, OCR request, editable text, and speech output
Chat.js	frontend/src/pages/	AAC assistant conversation and spoken replies
EmergencyPage.js	frontend/src/pages/	Emergency number list, add number, WhatsApp urgent message
DailyRoutinePage.js	frontend/src/pages/	Frequently spoken items from localStorage
dailyRoutine.js	frontend/src/utils/	Local routine tracking and threshold filtering
Auth.css	frontend/src/pages/	Login and signup styling
SubSubIconsPage.css	frontend/src/pages/	Shared card/grid styling for icon screens
DailyRoutinePage.css	frontend/src/pages/	Daily routine screen styling
ExpressDrawingPage.css	frontend/src/pages/	Drawing page styling

Frontend Process

1. React starts from index.js and renders App.
2. App wraps the route tree in AppProvider.
3. AppProvider reads loggedInUser from localStorage.
4. React Router maps URLs to the correct page.
5. Pages call api.js or auth.js to reach the backend.
6. Media URLs are normalized through normalizeMediaUrl so relative backend media paths resolve correctly.
7. User selections, generated sentences, playback state, and local history are handled inside the relevant page.
8. Long-term browser-side state is stored in localStorage for user session, routine items, selected ordering, and training attempts.

04 BACKEND API AND SERVER LAYER

The backend is an Express server that exposes REST endpoints, connects to PostgreSQL through Prisma, stores uploaded media, serves static assets, performs TTS, runs OCR, handles auth, and serves the React build in production.

Capabilities

EXPRESS API SERVER

index.js defines API routes for auth, TTS, OCR, chat, categories, icons, sub-icons, sub-sub-icons, emergency numbers, and speech attempts.

STATIC MEDIA DELIVERY

The backend serves backend/public through /public and supports uploaded media through /public/uploads.

FRONTEND BUILD SERVING

If frontend/build exists, the backend serves React pages for known frontend routes and falls back for browser page requests.

CORS CONFIGURATION

The backend allows localhost, LAN development hosts, Vercel, and Railway origins.

DATABASE ACCESS

Prisma Client reads and writes PostgreSQL tables for users, categories, icons, emergency numbers, speech attempts, and nested icon data.

FILE UPLOADS

Multer stores uploaded images and audio files in backend/public/uploads.

REMOTE FILE DOWNLOADS

node-fetch can download image/audio URLs and save them as backend uploads.

MEDIA FALLBACKS

subSubIconAudio.js resolves missing recordings using child audio, parent audio, group fallbacks, category fallbacks, and a default recording.

Backend Source Files

FILE	LOCATION	ROLE
index.js	backend/	Main Express server and all API endpoints
schema.prisma	backend/prisma/	Prisma database schema
seed.js	backend/prisma/	Database seeding script
data.js	backend/prisma/	Main categories, icons, sub-icons, emergency numbers, time periods
subSubIconData.js	backend/prisma/	Sub-sub-icon seed dataset
subSubIconAudio.js	backend/prisma/	Recording URL fallback resolver
migrations/*	backend/prisma/migrations/	Database migration history
public/	backend/	Images, recordings, category assets, uploads, and fallback media
package.json	backend/	Backend scripts and dependencies

Server Process

1. Backend starts with node index.js.
2. Environment values are loaded from backend/.env if present.
3. Express configures CORS, JSON parsing, static public files, upload fallback behavior, and frontend build serving.

4. Prisma connects to PostgreSQL using DATABASE_URL.

5. ensureAuthTables creates the User table if it does not exist.

6. REST endpoints process frontend requests and return JSON or audio responses.

7. Uploaded files and downloaded remote assets are stored under /public/uploads.

8. Server listens on process.env.PORT or 5551.

05 AUTHENTICATION AND USER SESSION

This module handles signup, login, password hashing, user profile serialization, and frontend session persistence.

Capabilities

SIGNUP

Users create an account with first name, last name, email, password, and patient condition.

LOGIN

Users log in using email and password.

PASSWORD HASHING

Passwords are hashed with crypto.scrypt and a random salt.

EMAIL NORMALIZATION

Backend lowercases and trims email before lookup and insert.

PATIENT TYPE VALIDATION

Backend accepts AUTISM, STROKE, ALZHEIMER, SPEECH_DELAY, and OTHER.

LOCAL FRONTEND SESSION

Frontend stores the returned user object in localStorage under loggedInUser.

Source Files

FILE	LOCATION	ROLE
Login.js	frontend/src/pages/	Login form and frontend validation
Signup.js	frontend/src/pages/	Signup form and patient type selection
auth.js	frontend/src/api/	Login/signup API requests
AppContext.js	frontend/src/context/	Saves user and exposes session state
index.js	backend/	Auth handlers, hashing, validation, and routes
schema.prisma	backend/prisma/	User model
20260502210000_add_users	backend/prisma/migrations/	User table migration

Backend Endpoints

ENDPOINT	VERB	DESCRIPTION
/api/auth/signup	POST	Creates a user account
/api/signup	POST	Alternate signup path
/signup	POST	Alternate signup path
/api/auth/login	POST	Logs in an existing user
/api/login	POST	Alternate login path

/login	POST	Alternate login path

Auth Process

1. User fills the signup form.
2. Frontend sends POST /api/auth/signup.
3. Backend validates fields, email format, and patient condition.
4. Backend checks if the email already exists.
5. Backend hashes the password with crypto.scrypt and stores passwordHash and salt.
6. User logs in through Login.js.
7. Backend verifies the submitted password against the stored hash.
8. Frontend stores the returned user in AppContext and localStorage.
9. App can read isLoggedIn and user from AppContext.

Alignment Note

Authentication currently stores the user object in localStorage. The visible source does not include JWT tokens, server sessions, refresh tokens, or route guards.

06 CATEGORY, ICON, SUB-ICON, AND SUB-SUB-ICON ENGINE

This module is the main AAC communication board. It organizes communication content into a hierarchy and lets the user navigate from broad categories into specific expression cards.

Capabilities

MAIN CATEGORIES

Loads top-level communication areas from /maincategories.

TIME PERIODS

Real Life Activities can route through time periods before showing icons.

ICON GRID

Displays icon cards with title, expression, and image.

SUB-ICON GRID

Displays sub-icons for a selected icon and allows multi-selection.

SUB-SUB-ICON GRID

Displays deeper nested content with search, selection, and parent context.

DETAIL PAGES

SubIconDetail and SubSubIconDetail show single-card views with speak controls.

MEDIA NORMALIZATION

Relative image and audio paths are converted to full backend URLs.

FALLBACK IMAGE

Missing or broken images fall back to /public/default.jpg.

Source Files

FILE	LOCATION	ROLE
MainCategoriesPage.js	frontend/src/pages/	Loads categories and routes special categories
TimePeriodsPage.js	frontend/src/pages/	Loads time periods for Real Life Activities
IconsPage.js	frontend/src/pages/	Loads icons by category or time period
SubIconsPage.js	frontend/src/pages/	Loads icon details and sub-icons
SubSubIconsPage.js	frontend/src/pages/	Loads nested sub-sub-icons and supports search
SubIconDetail.js	frontend/src/pages/	Shows one sub-icon
SubSubIconDetail.js	frontend/src/pages/	Shows one sub-sub-icon with audio controls
api.js	frontend/src/api/	normalizeMediaUrl and API base URL
index.js	backend/	Category/icon API routes
schema.prisma	backend/prisma/	MainCategory, TimePeriod, Icon, SubIcon, SubSubIcon models
data.js	backend/prisma/	Seed data for main hierarchy
subSubIconData.js	backend/prisma/	Seed data for nested hierarchy

Backend Endpoints

ENDPOINT	VERB	DESCRIPTION
/maincategories	GET	Returns all main categories
/maincategories/:id/timeperiods	GET	Returns time periods for a category
/timeperiods/:id/icons	GET	Returns icons for a time period
/maincategories/:id/icons	GET	Returns icons for a main category
/icons	GET	Returns icons, optionally filtered by category
/icons	POST	Creates an icon
/icons/:id	GET	Returns one icon with nested sub-icons

/subicons	GET	Returns sub-icons, optionally filtered by category
/icons/:iconId/subicons	GET	Returns sub-icons under an icon
/icons/:iconId/subicons/:subIconId	GET	Returns one sub-icon with parent icon
/subicons/:subIconId/subsubicons	GET	Returns sub-sub-icons under a sub-icon
/icons/:iconId/subicons/:subIconId/subsubicons/:id	GET	Returns one sub-sub-icon with parent context

Navigation Process

1. User opens Main Categories.
2. Frontend requests GET /maincategories.
3. MainCategoriesPage decides whether the category routes to a normal icon grid or a special module.
4. Normal category opens IconsPage and requests icons.
5. User chooses an icon and opens SubIconsPage.
6. SubIconsPage requests GET /icons/:id and receives the icon with subIcons.
7. If a sub-icon has subSubIcons, clicking it opens SubSubIconsPage.
8. If no nested data exists, clicking it opens SubIconDetail.
9. SubSubIconsPage lets the user search, select, and open individual details.

07 SPEECH GENERATION AND AUDIO PLAYBACK ENGINE

This module converts selected communication items into spoken output using human recordings, browser speech, Google TTS, or ElevenLabs AI voices.

Capabilities

VOICE MODE SELECTION

Users can select Human Records, Male, Female, Records with AI - Male, or Records with AI - Female.

HUMAN RECORDINGS

When voiceMode is human, the frontend plays stored audioUrl or recordingUrl values from backend/public.

BROWSER SPEECH

When voiceMode is male or female, the frontend uses window.speechSynthesis with Arabic voice selection and adjusted pitch/rate.

ELEVENLABS AI VOICES

When voiceMode is ai-male or ai-female, frontend sends text to /api/tts/speak and plays the returned MP3 blob.

GOOGLE TTS

The drawing and chat modules use /api/tts/gtts to generate speech from text.

SENTENCE GENERATION

SubIconsPage and SubSubIconsPage build sentences using time options, parent expressions, connectors, and selected item expressions.

QUEUE PLAYBACK

Human recordings are played in sequence for selected items.

AUDIO FALLBACKS

If an item has no direct audio, the app can use category audio, nested child audio, browser speech, or a backend fallback recording.

VOLUME AND SPEED CONTROL

SubSubIconDetail provides volume and playback speed sliders for human recordings.

Source Files

FILE	LOCATION	ROLE
api.js	frontend/src/api/	Browser speech, ElevenLabs request, Google TTS request, media URL helpers
SubIconsPage.js	frontend/src/pages/	Sentence generation, queue playback, voice mode, preload, fallback
SubSubIconsPage.js	frontend/src/pages/	Nested sentence generation and playback
SubIconDetail.js	frontend/src/pages/	Single sub-icon speech
SubSubIconDetail.js	frontend/src/pages/	Single sub-sub-icon speech, volume, speed
DailyRoutinePage.js	frontend/src/pages/	Speaks frequent items
Chat.js	frontend/src/pages/	Speaks assistant replies through Google TTS
ExpressDrawingPage.js	frontend/src/pages/	Speaks OCR text through Google TTS
index.js	backend/	/api/tts/speak and /api/tts/gtts endpoints
subSubIconAudio.js	backend/prisma/	Audio fallback resolver
public/records	backend/	Audio recordings
public/recordss	backend/	Audio recordings

Backend Endpoints

ENDPOINT	VERB	DESCRIPTION
/api/tts/speak	POST	Uses ElevenLabs API and returns audio/mpeg
/api/tts/gtts	POST	Uses google-tts-api and returns audio/mpeg

Playback Process

1. User selects one or more communication cards.
2. Frontend builds a sentence from selected expressions.
3. User chooses a voice mode.
4. If mode is Human Records, frontend plays existing recording URLs in order.
5. If an item recording is missing, frontend tries fallback audio or browser speech.
6. If mode is Male or Female, frontend uses the browser speechSynthesis API.
7. If mode is AI male/female, frontend requests POST /api/tts/speak.
8. Backend calls ElevenLabs and returns an MP3 buffer.
9. Frontend creates a blob URL and plays it with HTML5 Audio.
10. Drawing and chat use POST /api/tts/gtts for Google TTS.

TTS Tool Comparison

TOOL	BEST USE IN VOXI	ADVANTAGES
Human Recordings	Core AAC fixed vocabulary	Most predictable pronunciation, works from stored files, no per-request AI cost, strong for repeated phrases.
Browser Web Speech API	Male/Female quick speech	No backend endpoint required, no API key, very fast, useful fallback when recordings are missing.
ResponsiveVoice	Possible alternative browser	TTS Simple browser-facing API and broad voice catalog concept, but it is not currently wired in this codebase.
Google TTS via google-tts-api	Drawing and chat speech	Easy backend MP3 generation, simple request/response flow, good for short dynamic phrases.
ElevenLabs API	AI Male/Female voice modes	More natural voice quality, multilingual model support, stronger presentation for generated AAC sentences.

TOOL	DISADVANTAGES / LIMITS

Human Recordings	Requires manual recording and storage, cannot speak new sentences unless all parts have audio, quality depends on uploaded recordings.
Browser Web Speech API	Voice availability depends on browser/device, male/female selection is heuristic, pronunciation may vary between users.
ResponsiveVoice	Not used in the current source, may require external script/service availability and licensing checks before production use.
Google TTS via google-tts-api	Depends on internet access and an unofficial Google TTS access pattern, limited voice customization compared with ElevenLabs.
ElevenLabs API	Requires ELEVENLABS_API_KEY, has quota/cost considerations, adds external API latency and should not expose the key in frontend code.

Current Project Decision

The project currently combines human recordings, native browser speech, Google TTS, and ElevenLabs instead of relying on one TTS tool. This gives Voxi three useful fallback layers: stored recordings for stable AAC vocabulary, browser speech for instant local fallback, and backend-generated MP3 audio for dynamic drawing, chat, and AI voice output.

08 CONTENT CREATION AND MEDIA UPLOADS

This module allows new communication content to be added from the frontend and stored by the backend.

Capabilities

ADD SUB-ICON

SubIconsPage provides a modal for creating a new sub-icon under a selected icon.

IMAGE INPUT OPTIONS

Users can upload an image, paste an image URL, or capture a photo from a mobile camera.

AUDIO INPUT OPTIONS

Users can upload an audio file, paste an audio URL, or record audio through mobile capture.

IMAGE COMPRESSION

Camera images are compressed in the browser before upload to reduce file size.

FORMDATA SUBMISSION

Frontend sends title, expression, category, image, and audio as multipart FormData.

BACKEND FILE STORAGE

Multer stores uploaded files under backend/public/uploads.

REMOTE URL INGESTION

Backend can download remote image/audio URLs and store them as uploads.

IMMEDIATE UI UPDATE

After backend success, the new sub-icon is appended to frontend state without a full page reload.

Source Files

FILE	LOCATION	ROLE
SubIconsPage.js	frontend/src/pages/	Add SubIcon modal, camera capture, audio upload, FormData submission
index.js	backend/	Multer setup, upload routes, downloadFile helper
schema.prisma	backend/prisma/	SubIcon and SubSubIcon models
public/uploads	backend/	Stored uploaded files

Backend Endpoints

ENDPOINT	VERB	DESCRIPTION
/icons/:iconId/subicons	POST	Creates a sub-icon with uploaded or downloaded media
/subicons/:subIconId/subsubicons	POST	Creates a sub-sub-icon with uploaded or downloaded media

Content Creation Process

1. User opens a category icon and clicks Add SubIcon.
2. User enters title, expression, and category.
3. User chooses an image source: upload, URL, or camera.
4. User chooses an audio source: upload, URL, or record.
5. Frontend builds FormData.
6. Frontend sends POST /icons/:iconId/subicons.
7. Backend validates the parent icon.
8. Backend stores uploaded files or downloads remote file URLs.
9. Backend creates the SubIcon row in PostgreSQL.
10. Backend returns the created object with resolved recordingUrl.
11. Frontend adds it to the current card list.

09 SPEECH TRAINING AND PROGRESS TRACKING

This module helps users practice speaking a target word or phrase and compare their speech result to the expected text.

Capabilities

TARGET WORD INPUT

User enters a word or phrase to practice.

LANGUAGE SELECTION

Training supports Arabic Egypt, Arabic Saudi Arabia, English, French, and Spanish recognition language codes.

MICROPHONE RECORDING

MediaRecorder records the spoken attempt and provides audio playback.

SPEECH RECOGNITION

Browser SpeechRecognition converts the spoken attempt into text.

SCORING

Frontend calculates similarity using normalized text and Levenshtein distance.

PASSING SCORE

A score of 70 or higher is considered successful.

LOCAL HISTORY

Attempts are stored in localStorage per target phrase.

PROGRESS CHART

The page renders an SVG line chart for attempt scores.

Source Files

FILE	LOCATION	ROLE
TrainingPage.js	frontend/src/pages/	Speech recognition, recording, scoring, attempts table, chart
schema.prisma	backend/prisma/	SpeechAttempt model
index.js	backend/	SpeechAttempt endpoints
20260322110000...	backend/prisma/migrations/	SpeechAttempt table migration

Backend Endpoints

ENDPOINT	VERB	DESCRIPTION
/speech/attempts	GET	Returns saved speech attempts, optionally filtered by word
/speech/attempts	POST	Creates a speech attempt with word, transcript, and score

Training Process

1. User enters a target word or phrase.
2. User chooses recognition language.
3. User clicks Start Recording.
4. Frontend checks browser support, secure context, microphone permission, and online state.
5. MediaRecorder records the user's voice.
6. SpeechRecognition returns a transcript.

7. Frontend normalizes target and transcript.

8. Frontend calculates Levenshtein similarity score.

9. Attempt is stored in localStorage.

10. Progress chart and attempts table update.

Alignment Note

The backend includes /speech/attempts endpoints and a SpeechAttempt table, but TrainingPage currently stores attempts in localStorage and does not call those backend endpoints in the visible frontend source.

10 DRAWING OCR AND DRAWING-TO-SPEECH

This module lets users draw text or a simple expression, recognize it through OCR, edit the result, and speak it.

Capabilities

CANVAS DRAWING

ExpressDrawingPage provides a touch and mouse canvas for drawing.

IMAGE PREVIEW

The page mirrors the drawing into a preview canvas.

PNG DATA URL

Before OCR, the canvas is exported as a PNG data URL.

OCR PREPROCESSING

Backend crops around ink, adds padding, scales the drawing, thresholds dark pixels, and prepares a cleaner PNG.

ARABIC AND ENGLISH OCR

Backend chooses Arabic or English Tesseract data depending on selected language.

MULTI-PASS OCR

Backend tries different Tesseract page segmentation modes and returns the best text/confidence result.

EDIT BEFORE SPEAK

User can edit the recognized phrase before generating audio.

GOOGLE TTS OUTPUT

The final phrase is spoken using /api/tts/gtts.

Source Files

FILE	LOCATION	ROLE
ExpressDrawingPage.js	frontend/src/pages/	Canvas drawing, OCR request, editable phrase, speak action
ExpressDrawingPage.css	frontend/src/pages/	Drawing screen styling
api.js	frontend/src/api/	recognizeDrawing and speakDrawingText helpers
index.js	backend/	OCR parsing, preprocessing, Tesseract workers, endpoint
package.json	backend/	tesseract.js, language data, pngjs dependencies

Backend Endpoints

ENDPOINT	VERB	DESCRIPTION
/api/drawing/recognize	POST	Recognizes text from a PNG canvas image
/api/tts/gtts	POST	Speaks the recognized or edited text

Drawing Process

1. User draws on the canvas.
2. User clicks Run OCR.
3. Frontend paints a white background and exports the canvas as image/png base64.
4. Frontend sends POST /api/drawing/recognize with imageDataUrl and language.
5. Backend validates the data URL and PNG MIME type.
6. Backend reads the PNG with pngjs.
7. Backend finds ink bounds, crops, pads, scales, and thresholds the drawing.
8. Backend runs Tesseract OCR with the selected language data.
9. Backend returns text, confidence, and engine.
10. Frontend displays the recognized phrase for editing.
11. User clicks Speak Text.
12. Frontend requests /api/tts/gtts and plays the returned audio.

11 AAC ASSISTANT CHAT

This module provides a conversational assistant for AAC-related questions. It can use a local knowledge base or an optional Groq provider.

Capabilities

MULTI-LANGUAGE UI

Chat supports English, Arabic, French, and Spanish language modes.

QUICK PROMPTS

The page displays common AAC questions such as starting AAC, device refusal, and core words.

CHAT HISTORY

Frontend sends recent conversation history to the backend.

LOCAL KNOWLEDGE BASE

Backend includes local AAC intents and responses for common topics.

UNCLEAR INPUT DETECTION

Backend detects likely unclear messages and returns clarification replies.

OPTIONAL GROQ PROVIDER

If CHATBOT_PROVIDER=groq and GROQ_API_KEY is configured, backend calls Groq's OpenAI-compatible chat endpoint.

SPOKEN REPLIES

Frontend can generate audio for the latest assistant reply using Google TTS.

Source Files

FILE	LOCATION	ROLE
Chat.js	frontend/src/pages/	Chat UI, quick prompts, message list, spoken replies
api.js	frontend/src/api/	sendChatMessage and generateGoogleTtsAudioUrl
index.js	backend/	Chat prompt, local knowledge base, Groq integration, chat endpoint

Backend Endpoints

ENDPOINT	VERB	DESCRIPTION
/api/chat	POST	Sends a message to the AAC assistant
/chat	POST	Alternate chat path
/api/tts/gtts	POST	Speaks assistant replies

Chat Process

1. User opens AAC Assistant.
2. User chooses a language.
3. User types a question or clicks a quick prompt.
4. Frontend sends POST /api/chat with message, language, and history.
5. Backend sanitizes history and validates the message.
6. If Groq is configured, backend requests the external provider.
7. If no external provider is active, backend matches the local knowledge base.
8. If input is unclear, backend returns a clarification response.
9. Frontend displays the assistant reply.
10. User can click Speak to generate Google TTS audio for the reply.

12 EMERGENCY AND DAILY ROUTINE SUPPORT

This module adds practical support tools around communication: emergency contacts and frequently used routine phrases.

Capabilities

EMERGENCY NUMBER LIST

EmergencyPage loads saved emergency numbers from the backend.

ADD EMERGENCY NUMBER

Users can add a number and label through a modal.

MULTI-LABEL INPUTS

The frontend includes English, Arabic, French, and Spanish label inputs, then sends the best available label.

URGENT WHATSAPP MESSAGE

User can choose a number, write a message, and open a wa.me link.

DAILY ROUTINE TRACKING

Repeatedly spoken sub-icons and sub-sub-icons are tracked in localStorage.

ROUTINE THRESHOLD

Items spoken 3 or more times appear in Daily Routine.

ROUTINE PLAYBACK

DailyRoutinePage can speak one routine item or the entire routine list.

Source Files

FILE	LOCATION	ROLE
EmergencyPage.js	frontend/src/pages/	Emergency numbers and WhatsApp urgent messages
DailyRoutinePage.js	frontend/src/pages/	Frequently spoken item display and playback
dailyRoutine.js	frontend/src/utils/	Local routine tracking logic
SubIconsPage.js	frontend/src/pages/	Tracks selected item playback
SubSubIconsPage.js	frontend/src/pages/	Tracks nested item playback
SubIconDetail.js	frontend/src/pages/	Tracks single sub-icon playback
SubSubIconDetail.js	frontend/src/pages/	Tracks single sub-sub-icon playback
index.js	backend/	Emergency number endpoints
schema.prisma	backend/prisma/	EmergencyNumber model

Backend Endpoints

ENDPOINT	VERB	DESCRIPTION
/emergency-numbers	GET	Returns emergency numbers
/emergency-numbers	POST	Creates or updates an emergency number by number

Emergency Process

1. EmergencyPage requests GET /emergency-numbers.
2. Backend returns numbers from PostgreSQL.
3. User can open Add Emergency Number.
4. Frontend sends POST /emergency-numbers.
5. Backend upserts by number.
6. Frontend updates the number list.
7. User can open Send Urgent Message.
8. Frontend converts the phone number for WhatsApp and opens wa.me with encoded text.

Daily Routine Process

1. User speaks an item from a category page or detail page.
2. trackRoutinePlayback stores or updates the item in localStorage.
3. speakCount increments every time the item is spoken.
4. Items with speakCount >= 3 appear in DailyRoutinePage.
5. User can speak one item, speak all routine items, remove an item, or clear the routine.

13 DATABASE MODEL AND SEEDING

This module defines and populates the structured communication content used by the frontend.

Prisma Models

MODEL	PURPOSE
MainCategory	Top-level AAC category
TimePeriod	Time grouping for Real Life Activities
Icon	Main icon under a category or time period
SubIcon	Child icon under an Icon
SubSubIcon	Deeper child icon under a SubIcon
EmergencyNumber	Stored emergency contact number and label
SpeechAttempt	Backend-supported speech practice attempt
User	Application user account

Key Relationships

1. MainCategory has many Icons.
2. MainCategory has many TimePeriods.
3. TimePeriod has many Icons.
4. Icon has many SubIcons.
5. SubIcon has many SubSubIcons.
6. User stores authentication and patient condition data.
7. EmergencyNumber and SpeechAttempt are standalone support tables.

Migrations

MIGRATION	DESCRIPTION
20260304212232_init	Initial MainCategory, Icon, and SubIcon tables

20260312004925_add_audio_url	Adds audioUrl to Icon
20260322110000_add_time_periods_emergency_speech	Adds TimePeriod, EmergencyNumber, SpeechAttempt, and Icon timePeriodId
20260427120000_add_subsubicons	Adds SubSubIcon and unique nested constraints
20260502210000_add_users	Adds User table and unique email

Seed Process

1. npm run seed runs node prisma/seed.js.
2. seed.js upserts main categories.
3. seed.js upserts time periods.
4. seed.js upserts emergency numbers.
5. seed.js upserts icons and attaches them to main categories and time periods.
6. seed.js combines all sub-icon arrays from data.js.
7. seed.js upserts sub-icons under the correct parent icon.
8. seed.js reads subSubIconsData.
9. seed.js finds each parent SubIcon by title and category.
10. seed.js upserts sub-sub-icons and resolves audio URLs.

Important Data Files

FILE	LOCATION	ROLE
data.js	backend/prisma/	Main seed dataset
subSubIconData.js	backend/prisma/	Deep nested seed dataset
subSubIconAudio.js	backend/prisma/	Audio URL fallback rules

public/categories	backend/	Category images
public/timeperiods	backend/	Time period images
public/uploads	backend/	Runtime uploaded media
public/records	backend/	Human audio recordings
public/recordss	backend/	Human audio recordings
public/* category folders	backend/	Image assets for icons and sub-icons

14 DEPLOYMENT AND API CONFIGURATION

This module documents how the frontend and backend decide where to send requests and how the deployed app is served.

Capabilities

API BASE URL RESOLUTION

frontend/src/api/api.js checks REACT_APP_API_BASE_URL or REACT_APP_API_URL first.

PUBLIC BACKEND FALLBACK

If no safe explicit URL exists, the frontend uses https://tts-production-6e70.up.railway.app.

LOCALHOST SAFETY

The frontend avoids using a localhost API URL when the browser is running on a public hostname.

LEGACY URL NORMALIZATION

normalizeMediaUrl rewrites old local or legacy backend media URLs to the current API base URL.

RAILWAY BACKEND

The backend is written to run on process.env.PORT or port 5551.

VERCEL REWRITE

frontend/vercel.json rewrites /backend/* to a Railway backend URL.

CORS ALLOWLIST

Backend allows local development origins and deployed Vercel/Railway origins.

Production Serving

1. The backend checks if frontend/build/index.html exists.
2. If it exists, Express serves static frontend build files.
3. Known frontend routes are sent back to index.html.
4. API routes still respond normally.

Alignment Notes

FRONTEND TRANSLATION ROUTE

api.js includes translateText calling POST /api/translate, and Translator.js exists but is commented out. The visible backend source does not currently define /api/translate. This endpoint should be added or the unused translation UI should remain disabled.

DEPLOYMENT URLS

frontend/src/api/api.js points to https://tts-production-6e70.up.railway.app. backend/index.js CORS also references https://tts-production-77b9.up.railway.app and frontend/vercel.json rewrites to that URL. These deployment URLs should be verified so the frontend, backend, and Vercel rewrite point to the intended production backend.

SPEECH ATTEMPT STORAGE

The backend supports SpeechAttempt persistence, but TrainingPage currently stores attempts locally. If persistent training history is required, TrainingPage should call GET and POST /speech/attempts.

AUTH SECURITY

The current auth implementation is suitable for basic app state but does not include token-based authorization or protected backend routes.

15 OVERALL BACKEND ENDPOINT MAP

ENDPOINT	VERB	MODULE
/health	GET	Server health check
/api/auth/signup	POST	Authentication
/api/signup	POST	Authentication alias
/signup	POST	Authentication alias
/api/auth/login	POST	Authentication
/api/login	POST	Authentication alias
/login	POST	Authentication alias
/api/tts/speak	POST	ElevenLabs TTS
/api/tts/gtts	POST	Google TTS
/api/chat	POST	AAC Assistant
/chat	POST	AAC Assistant alias

/api/drawing/recognize	POST	Drawing OCR
/icons	GET	Icons
/icons	POST	Icon creation
/icons/:iconId/subicons	POST	Sub-icon creation
/subicons	GET	Sub-icons
/icons/:iconId/subicons	GET	Sub-icons by icon
/maincategories	GET	Main categories
/maincategories/:id/icons	GET	Icons by main category
/icons/:id	GET	One icon
/icons/:iconId/subicons/:subIconId	GET	One sub-icon
/subicons/:subIconId/subsubicons	POST	Sub-sub-icon creation
/subicons/:subIconId/subsubicons	GET	Sub-sub-icons by sub-icon
/icons/:iconId/subicons/:subIconId/subsubicons/:id	GET	One sub-sub-icon
/maincategories/:id/timeperiods	GET	Time periods
/timeperiods/:id/icons	GET	Icons by time period
/emergency-numbers	GET	Emergency numbers
/emergency-numbers	POST	Emergency number upsert
/speech/attempts	GET	Speech attempts
/speech/attempts	POST	Speech attempt creation

16 WHAT WAS DONE IN THE FRONTEND

FRONTEND IMPLEMENTATION SUMMARY

1. Built a React single page application with route-based pages.
2. Added global user session state through AppContext.
3. Added login and signup screens connected to backend auth endpoints.
4. Added a main category screen that loads backend categories and routes special modules.
5. Added icon, sub-icon, and sub-sub-icon pages.
6. Added multi-select communication cards.
7. Added sentence generation using time options, connectors, parent text, and selected expressions.
8. Added voice mode controls for human records, browser voices, and AI voices.
9. Added sequential audio playback for selected recordings.
10. Added fallback to browser speech when recordings are missing or fail.
11. Added sub-icon creation modal with image upload, image URL, camera capture, audio upload, audio URL, and audio recording.
12. Added image compression before camera uploads.
13. Added daily routine tracking based on repeated playback.
14. Added speech training with browser recognition, MediaRecorder audio playback, Levenshtein scoring, and progress chart.
15. Added drawing canvas, OCR request, editable recognized text, and speech output.
16. Added AAC Assistant chat UI with quick prompts, language selection, chat history, and spoken replies.
17. Added emergency number list, number creation modal, and WhatsApp urgent message flow.
18. Added API URL normalization for deployed and local environments.
19. Added fallback image handling for broken media.

17 WHAT WAS DONE IN THE BACKEND

BACKEND IMPLEMENTATION SUMMARY

1. Built an Express backend using ES modules.
2. Configured CORS for local and deployed frontend origins.
3. Configured static serving for backend/public and frontend/build.
4. Added fallback handling for uploaded image paths and default images.
5. Connected PostgreSQL through Prisma Client.
6. Created Prisma models for users, categories, time periods, icons, sub-icons, sub-sub-icons, emergency numbers, and speech attempts.
7. Added Prisma migrations for the schema evolution.
8. Added seed script for categories, time periods, emergency numbers, icons, sub-icons, and sub-sub-icons.
9. Added audio fallback resolver for missing recordings.
10. Added signup and login endpoints with crypto.scrypt password hashing.
11. Added ElevenLabs TTS endpoint.
12. Added Google TTS endpoint.
13. Added OCR endpoint using Tesseract.js and PNG preprocessing.
14. Added optional Groq chatbot integration and local AAC knowledge base fallback.
15. Added category/icon/sub-icon/sub-sub-icon read endpoints.
16. Added sub-icon and sub-sub-icon creation endpoints with Multer uploads.
17. Added remote media download helper for image/audio URL inputs.
18. Added emergency number GET and POST endpoints.
19. Added speech attempt GET and POST endpoints.
20. Added health check endpoint.

18 VERIFICATION CHECKLIST

Prerequisites

1. Backend has a valid DATABASE_URL.
2. Backend dependencies are installed with npm install inside backend.
3. Frontend dependencies are installed with npm install inside frontend.
4. Database migrations are applied.
5. Seed data is loaded with npm run seed from backend if needed.
6. ELEVENLABS_API_KEY is configured if AI voice modes are required.
7. GROQ_API_KEY is configured only if CHATBOT_PROVIDER=groq is required.
8. Browser microphone permission is enabled for training.
9. OCR and TTS tests should be done with a working internet connection.

Module Verification

#	ACTION	EXPECTED OUTCOME
1.1	Open /	Landing page renders
1.2	Open /signup	Signup form appears
1.3	Create a valid account	Backend creates User and frontend redirects to login
1.4	Login with valid account	User is saved in AppContext/localStorage
2.1	Open /main-categories	Categories load from /maincategories
2.2	Click Real Life Activities	Time periods load from /maincategories/:id/timeperiods
2.3	Click a normal category	Icons load from /maincategories/:id/icons
2.4	Click an icon	Sub-icons load from /icons/:id
2.5	Click a sub-icon with children	Sub-sub-icons page opens
2.6	Search in SubSubIconsPage	List filters by title/expression
3.1	Select multiple sub-icons	Sentence preview updates

3.2	Play with Human Records	Stored audio plays in sequence
3.3	Play with Male/Female	Browser speech plays generated sentence
3.4	Play with AI voice	/api/tts/speak returns audio and frontend plays it
3.5	Open SubSubIconDetail	Volume and speed sliders affect playback
4.1	Add SubIcon with uploaded image/audio	Backend stores files in /public/uploads and returns created row
4.2	Add SubIcon with remote URLs	Backend downloads files and creates row
4.3	Add SubIcon with camera/mic capture	Captured files upload successfully
5.1	Open TrainingPage	Training UI renders
5.2	Record a target phrase	Transcript and score appear
5.3	Repeat same target phrase	Attempts table and chart update
5.4	Clear current attempts	Local attempts for the word are removed
6.1	Open ExpressDrawingPage	Canvas renders
6.2	Draw text and run OCR	/api/drawing/recognize returns recognized text
6.3	Edit recognized text and speak	/api/tts/gtts returns audio and frontend plays it
7.1	Open AAC Assistant	Chat UI renders
7.2	Send a quick prompt	/api/chat returns a local or Groq reply
7.3	Click Speak	Latest assistant reply plays as Google TTS
8.1	Open EmergencyPage	/emergency-numbers data renders
8.2	Add emergency number	Backend upserts number and frontend list updates
8.3	Send urgent message	Browser opens WhatsApp wa.me link
9.1	Speak the same item 3 times	Item appears in Daily Routine
9.2	Speak all routine items	Routine items play in order
9.3	Remove routine item	Item disappears from Daily Routine
9.4	Clear routine	Daily Routine list becomes empty
10.1	Open deployed frontend	API calls use configured production backend
10.2	Open direct frontend route in backend build	Backend returns index.html for React route

19 SOURCES REVIEWED

The report was prepared from the current local project source code.

Frontend files reviewed:

- frontend/package.json
- frontend/vercel.json
- frontend/src/App.js
- frontend/src/index.js
- frontend/src/context/AppContext.js
- frontend/src/api/api.js
- frontend/src/api/auth.js
- frontend/src/pages/Login.js
- frontend/src/pages/Signup.js
- frontend/src/pages/MainCategoriesPage.js
- frontend/src/pages/TimePeriodsPage.js
- frontend/src/pages/IconsPage.js
- frontend/src/pages/SubIconsPage.js
- frontend/src/pages/SubSubIconsPage.js
- frontend/src/pages/SubIconDetail.js
- frontend/src/pages/SubSubIconDetail.js
- frontend/src/pages/TrainingPage.js

frontend/src/pages/ExpressDrawingPage.js

frontend/src/pages/Chat.js

frontend/src/pages/EmergencyPage.js

frontend/src/pages/DailyRoutinePage.js

frontend/src/utils/dailyRoutine.js

frontend/src/components/Translator.js

frontend/src/components/SpeakButton.js

Backend files reviewed:

backend/package.json

backend/index.js

backend/prisma/schema.prisma

backend/prisma/seed.js

backend/prisma/data.js

backend/prisma/subSubIconData.js

backend/prisma/subSubIconAudio.js

backend/prisma/migrations/*

backend/public/*